FAST National University of Computer and Emerging Sciences, Lahore

Artificial Intelligence — Semester Project

Resumatch

AI-Powered Resume–Job Matching Platform

Deliverable 2: Model Architecture & Empirical Evaluation

Track A — Application Development

Member	Roll No. / Section
Muhammad Sufyan	24L-2601 / BSDS-5B
Muaaz Fahad	24L-2563 / BSDS-5B
Hanan Ishaque	24L-2537 / BSDS-5B

November 2026

Contents

1 Introduction

Phase 3: Model Architecture & Training

2 System Overview
3 Stage 1: Bi-encoder
4 Stage 2: MatchNet
5 Baselines
6 Hyperparameter Tuning
7 Training Curves

Phase 4: Evaluation & Analysis

8 Validation Strategy
9 Metrics and Why
10 Results
11 Error Analysis
12 Bias Check
13 Limitations
14 Reproducibility and Model Weights
15 Contribution Table
16 Appendix: Full MatchNet Grid

References

1 Introduction

Deliverable 1 defined the problem (a job seeker uploads a resume and receives the three best open postings, each with an apply link and an ATS-style score), collected and cleaned 2,632 resumes and 28,088 postings, and built a rule-based ATS score and 15,755 labelled resume–posting pairs. This deliverable covers Phases 3 and 4: it trains the matching models, tunes them, and evaluates them on held-out data, including an error analysis and a bias check.

Two models are trained, both in PyTorch:

- a **bi-encoder** (MiniLM-L6 sentence transformer, fine-tuned contrastively) that embeds resumes and postings in one vector space, so that all open postings can be ranked by cosine similarity in milliseconds;
- **MatchNet**, a small neural network that re-scores the bi-encoder's shortlist from the two embeddings plus ten interpretable features (TF-IDF similarity, skill coverage, years of experience, title match, the ATS score, and others) and outputs a match probability.

Both are compared with five baselines (the rule-based ATS score, TF-IDF cosine, the pretrained encoder, logistic regression and gradient boosting, the last two from scikit-learn).

Two evaluation tasks. Our own pairs from Deliverable 1 are *weakly* labelled: a pair is positive when the resume and posting share an occupation category. A model can score well on them by recognizing occupations, which is necessary but not sufficient for a good match. We therefore also evaluate on an external, *human-labelled* dataset (`cnamuangtoun/resume-job-description-fit`), where human annotators labelled 8,000 pairs as No Fit, Potential Fit or Good Fit. Task A (weak labels) measures occupation-level matching on our own data; Task B (human labels) measures whether the model agrees with human judgement of fit within and across occupations, and is the harder and more meaningful test.

Phase 3: Model Architecture & Training

2 System Overview

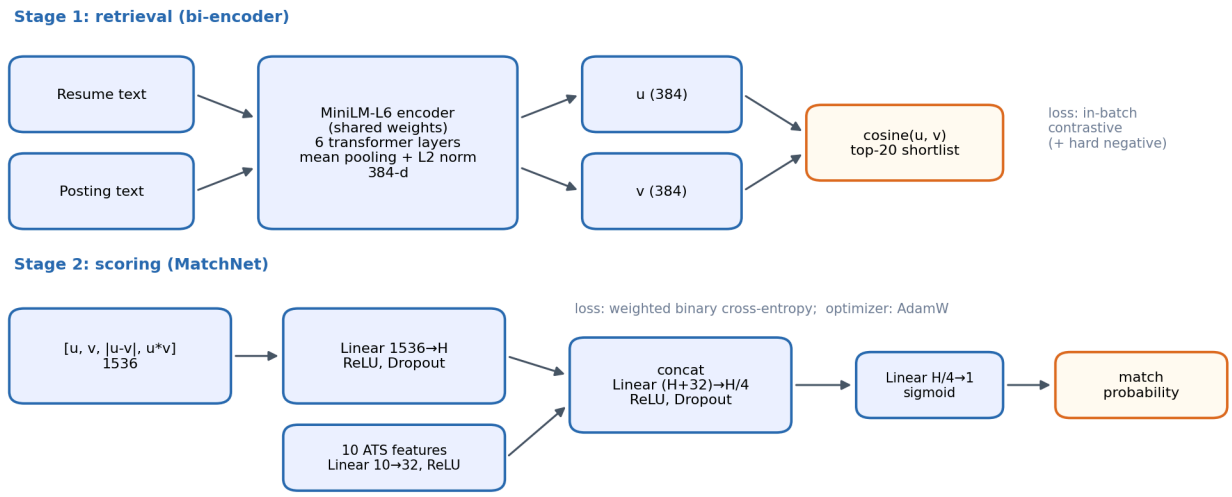


Figure 1: Model architecture. Stage 1: the bi-encoder embeds the resume and every open posting and keeps the 20 closest postings. Stage 2: MatchNet re-scores each shortlisted pair from the two embeddings and ten scalar features; the top three are returned with their ATS score and apply link.

The two-stage design follows the product requirement. Ranking thousands of postings with a model that reads both documents jointly would be slow, so stage 1 uses independent embeddings that can be pre-computed for every posting. Stage 2 only sees 20 candidates and can afford richer inputs, including the explicit skill and experience features that make the result explainable to the user. Table 1 lists the frameworks used.

Component	Implementation
Bi-encoder	PyTorch 2.8 training loop around `sentence-transformers` (MiniLM-L6), `src/models/bi_encoder.py`
MatchNet	PyTorch `nn.Module`, `src/models/matchnet.py`
Baselines	scikit-learn 1.6 (`LogisticRegression`, `HistGradientBoostingClassifier`, TF-IDF)
Metrics	scikit-learn metrics with bootstrap confidence intervals, `src/evaluate.py`
Hardware	Apple M1 Pro (16 GB), PyTorch MPS backend; seed 42 everywhere

Table 1: Frameworks and where each component lives in the repository.

3 Stage 1: Bi-encoder

Layers. `all-MiniLM-L6-v2`: a 6-layer transformer (hidden size 384, 12 attention heads, 22.7M parameters), inputs truncated to 256 word pieces, followed by mean pooling over tokens and L2 normalization. Resumes and postings share the same weights.

Loss. Multiple-negatives ranking loss. A batch holds B triplets (resume, matching posting, hard negative posting); hard negatives are postings from a related but different occupation, mined in Deliverable 1. Each resume must pick its own posting among all 2B postings in the batch, so the other resumes' positives act as additional negatives: $L = \text{CrossEntropy}(s \cdot \cos(r_i, p_j), i)$ with scale $s = 20$. This uses every posting in the batch as a negative at no extra cost and directly optimizes ranking, which is what stage 1 is for.

Optimizer. AdamW (weight decay 0.01), linear warm-up over the first 10% of steps then linear decay to zero, gradient-norm clipping at 1.0, gradient checkpointing to fit the 16 GB machine. Each epoch draws one triplet per training resume (1,842 triplets), cycling through that resume's positives and hard negatives across epochs.

4 Stage 2: MatchNet

Layers. For resume embedding u and posting embedding v (384-d each, from the fine-tuned bi-encoder) MatchNet builds the standard interaction vector $[u, v, |u-v|, u \odot v]$ (1,536 values) and passes it through

- an embedding branch: Linear($1536 \to H$), ReLU, Dropout;
- a feature branch: Linear($10 \to 32$), ReLU, on the standardized scalar features;
- a head on the concatenation: Linear($H+32 \to H/4$), ReLU, Dropout, Linear($H/4 \to 1$), sigmoid.

The selected models have $H = 256$ (Task A, 412,385 parameters) and $H = 128$ (Task B, 202,273 parameters).

Loss. Binary cross-entropy on the logit (`BCEWithLogitsLoss`) with `pos_weight` = negatives / positives in the training split, so that both classes contribute equally to the gradient despite the class imbalance.

Optimizer. AdamW (weight decay 10^{-4}), at most 60 epochs, early stopping when validation AUC-ROC has not improved for 6 epochs; the weights of the best epoch are restored.

5 Baselines

- **ATS score (rules)**: the Deliverable 1 score (skill coverage, experience, title and structure), used as a classifier score directly.
- **TF-IDF cosine** and **MiniLM cosine (pretrained)**: similarity without any training on our data; **bi-encoder cosine (fine-tuned)** isolates the gain from stage 1 alone.
- **Logistic regression** and **gradient boosting** (scikit-learn) on the same ten scalar features plus both cosine similarities, tuned on the validation split. They answer whether a neural network is needed at all.

6 Hyperparameter Tuning

All tuning uses the validation split only; the test split is touched once, for the final numbers.

Bi-encoder. Fine-tuning a transformer is expensive (about 3 minutes per epoch here), so instead of a full grid we varied one parameter at a time around the usual sentence-transformer defaults (learning rate 3×10^{-5}, batch 32, length 256): learning rate (three values a factor of three apart), batch size (which also sets the number of in-batch negatives) and maximum input length, each for 3 epochs, 5 runs in total (Table 2). The selection criterion is the AUC of positives against *hard* negatives, because separating related occupations is the difficult part; random negatives are already easy for the pre-trained model.

Learning rate	Batch	Max len	Best ep.	Val AUC	Val AUC (hard)	Val P@3	Min
pretrained MiniLM, no fine-tuning				0.644	0.539	0.429	–
1×10^{-5}	32	256	3	0.729	0.628	0.544	8
3×10^{-5}	32	256	3	0.771	0.683	0.609	8
1×10^{-4}	**32**	**256**	**3**	**0.801**	**0.724**	**0.631**	**8**
3×10^{-5}	16	256	3	0.793	0.714	0.629	9
3×10^{-5}	32	128	3	0.751	0.666	0.579	3

Table 2: Bi-encoder hyperparameter search (validation split). Selection criterion: AUC of positives vs. hard negatives. The selected run is in bold.

The selected configuration (learning rate 1×10^{-4}, batch 32, length 256, epoch 3) raised validation AUC on hard negatives from 0.539 to 0.724 and precision@3 from 0.429 to 0.631. Three observations justify the choice:

- *Learning rate* mattered most. With only 3 epochs, 10^{-5} barely moves the pretrained weights; 10^{-4} adapted fastest and showed no sign of damaging the encoder (validation AUC rose every epoch).
- *Batch size*: 16 beat 32 at the same learning rate. With a fixed number of epochs, a smaller batch takes twice as many update steps, which outweighed having fewer in-batch negatives.
- *Input length*: 128 word pieces is clearly worse than 256 (the median cleaned resume has 745 words, so 128 pieces cut off most of the experience section) and was kept at 256.

Every run was still improving at its last epoch, so longer training (and combining 10^{-4} with batch 16) is the obvious next step; it was limited by compute, not by overfitting.

MatchNet. MatchNet trains in seconds, so we ran a full grid of 36 combinations per task: learning rate 10^{-3}, 3×10^{-4}, 10^{-4}, batch size 64, 256, dropout 0.1, 0.3, 0.5 and hidden size 128, 256. Table 3

summarizes the effect of each value; the full grid is in Section 16.

Hyperparameter	Value	Task A: mean / max val AUC	Task B: mean / max val AUC
Learning rate	1×10^{-4}	0.835 / 0.839	0.773 / 0.781
	3×10^{-4}	0.841 / 0.847	0.781 / 0.793
	1×10^{-3}	**0.853 / 0.860**	**0.794 / 0.807**
Batch size	64	**0.845 / 0.860**	**0.787 / 0.807**
	256	0.840 / 0.857	0.779 / 0.800
Dropout	0.1	0.841 / 0.858	0.780 / 0.795
	0.3	**0.843 / 0.860**	0.781 / 0.807
	0.5	0.845 / 0.858	**0.788 / 0.807**
Hidden size H	128	0.842 / 0.855	**0.784 / 0.807**
	256	**0.844 / 0.860**	0.782 / 0.806

Table 3: MatchNet grid search: all 36 combinations of the values below were trained per task; each cell averages (and maximizes) validation AUC over the runs with that value. Bold marks the value of the selected configuration. The full grid is in the appendix.

Selected: Task A learning rate 1×10^{-3}, batch 64, dropout 0.3, $H = 256$ (val AUC 0.860, best epoch 14); Task B learning rate 1×10^{-3}, batch 64, dropout 0.5, $H = 128$ (val AUC 0.807, best epoch 26). The largest learning rate won on both tasks: with early stopping on validation AUC, a fast optimizer is not at risk of overfitting, and the network is small. Task B has only 5,463 training pairs, about half of Task A, which is why the smaller network with the strongest dropout was selected there. Batch size mattered little.

Ablations. Two questions were checked on the validation split with the selected configuration. *Does fine-tuning the encoder help MatchNet?* With pretrained instead of fine-tuned embeddings, validation AUC is 0.783 (Task A) and 0.820 (Task B), against 0.860 and 0.807. *Do the explicit ATS features help?* Without the feature branch it is 0.834 (Task A) and 0.800 (Task B), so the interpretable features add a small but consistent gain on top of the embeddings. Fine-tuning helps strongly on Task A but *not* on Task B, where pretrained embeddings are slightly better. The bi-encoder was fine-tuned on Task A's occupation labels; it learned to separate occupations, which matters less when most Task B pairs already share an occupation and differ in seniority and specific skills. We keep one encoder for both tasks because the product uses one, and report this as a limitation.

7 Training Curves

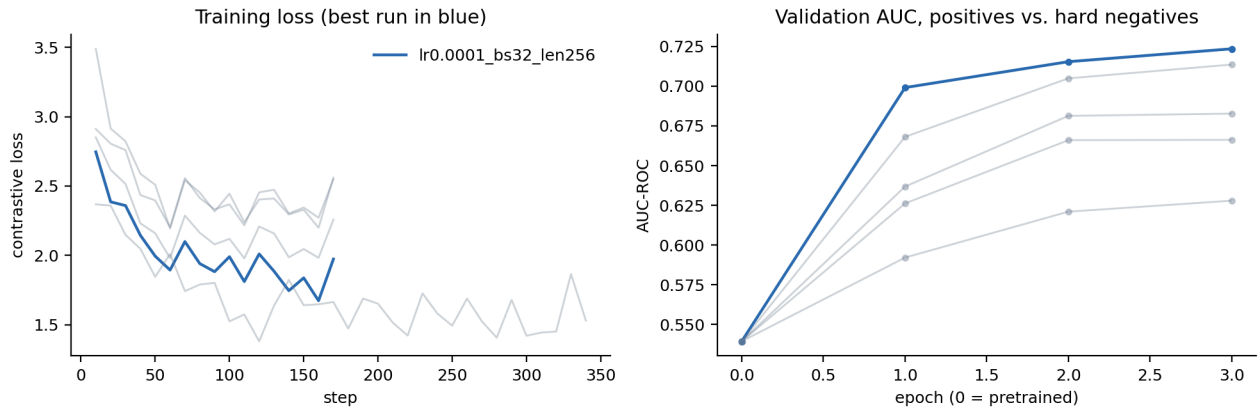


Figure 2: Bi-encoder fine-tuning: training loss per 10 steps (left) and validation AUC of positives vs. hard negatives after each epoch (right; epoch 0 is the pretrained model). The selected run is in blue, the other grid runs in grey.

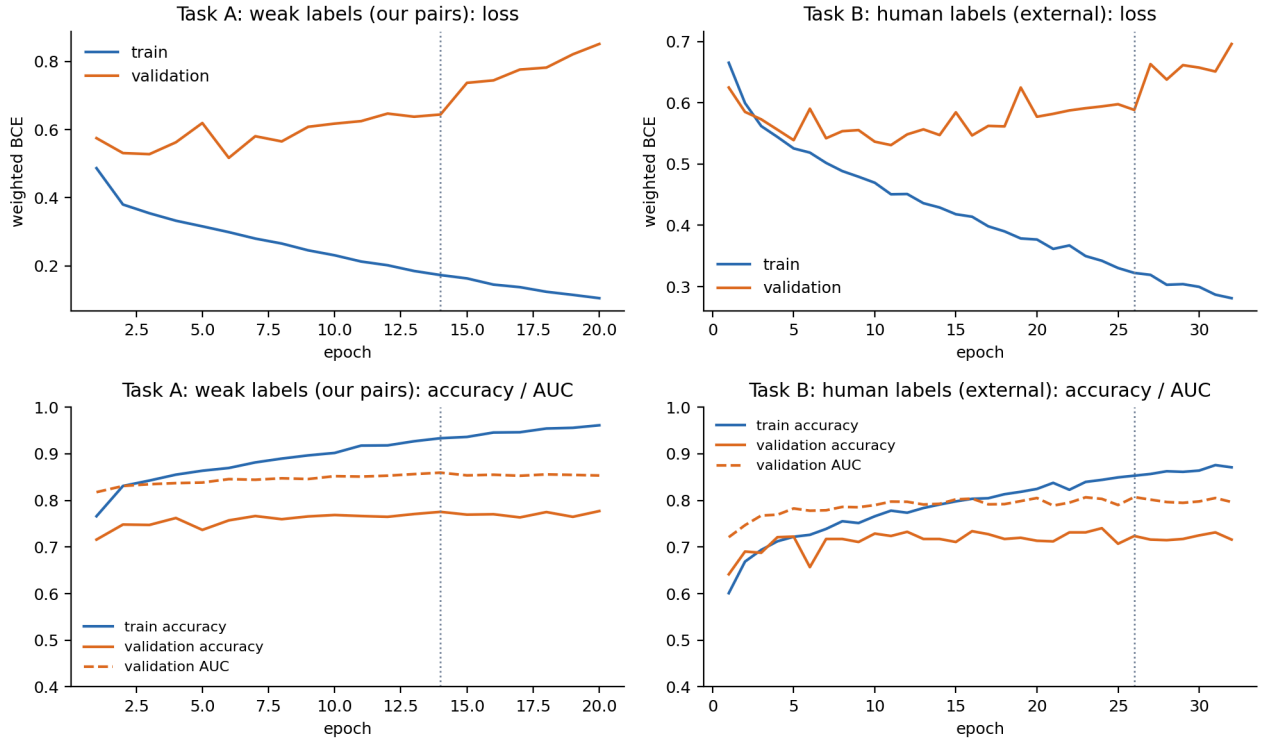


Figure 3: MatchNet training for the selected configurations: loss (top) and accuracy (bottom) on the training and validation splits. The dashed line marks the early-stopping epoch whose weights are kept.

Validation loss starts rising after a few epochs while validation AUC is still flat or improving: the network keeps ranking pairs correctly but becomes over-confident on the pairs it gets wrong, which cross-entropy punishes heavily. This is why early stopping monitors AUC, the quantity the product depends on (ranking), rather than loss; the train–validation gap in accuracy also shows that dropout and early stopping are needed, since MatchNet can fit the training pairs almost perfectly.

Phase 4: Evaluation & Analysis

8 Validation Strategy

We use a fixed train / validation / test split rather than K-fold cross-validation. Fine-tuning the transformer five or ten times per configuration would multiply a one-hour search, and with thousands of pairs a single held-out split already gives tight confidence intervals (reported below). The splits are made by *entity*, not by pair, so that no information leaks:

- **Task A**: resumes and postings were assigned to train / validation / test in Deliverable 1 (70/15/15 by resume, stratified by category); a pair belongs to the split of its resume and posting. Pairs: 11,052 train, 2,351 validation, 2,352 test.
- **Task B**: the dataset ships with a train and a test file whose job descriptions are disjoint. We keep its test file as our test split and carve the validation split out of its train file by job description (15% of job descriptions, `GroupShuffleSplit`), so validation also measures generalization to unseen postings.

The decision threshold of every model is chosen on the validation split (maximum F1) and applied unchanged to the test split. 95% confidence intervals come from 1,000 bootstrap resamples of the test pairs.

9 Metrics and Why

Both test splits happen to be close to balanced (50% and 51% positive), but only by construction: in the product, almost every posting is a non-match for a given resume, so the deployed problem is heavily imbalanced. Accuracy rewards predicting the majority class and would look good on the deployed problem even for a useless model, so it is reported only for completeness. We report:

- **Precision**: of the postings we recommend, how many are real matches. Low precision wastes the user's applications and is the product's main failure mode.
- **Recall**: of the real matches, how many we find. Low recall hides good jobs.
- **F1**: their harmonic mean, the threshold-dependent summary.
- **AUC-ROC**: probability that a random positive pair scores above a random negative one. It is threshold-free and insensitive to class balance, so it is our primary metric for ranking quality.
- **PR-AUC** (average precision): complements AUC-ROC when positives are rare, because it ignores the large pool of easy true negatives.

For the product itself we also report **precision@3**: the share of the three recommended postings that are relevant.

10 Results

Model	Precision	Recall	F1	Accuracy	AUC-ROC [95% CI]	PR-AUC
ATS score (rules)	0.498	0.997	0.665	0.500	0.588 [0.56, 0.61]	0.563
TF-IDF cosine	0.552	0.973	0.704	0.594	0.571 [0.55, 0.59]	0.555
MiniLM cosine (pretrained)	0.540	0.939	0.686	0.574	0.661 [0.64, 0.68]	0.622
Bi-encoder cosine (fine-tuned)	0.639	0.876	0.739	0.694	0.790 [0.77, 0.81]	0.770
Logistic regression	0.659	0.864	0.748	0.711	0.808 [0.79, 0.82]	0.791
Gradient boosting	0.724	0.810	0.764	0.752	0.832 [0.82, 0.85]	0.818
MatchNet	**0.705**	**0.883**	**0.784**	**0.759**	**0.862 [0.85, 0.88]**	**0.855**

Table 4: Task A (weak labels): test-set results (n = 2,352, 50% positive). Thresholds were chosen on the validation split by maximum F1.

Model	Precision	Recall	F1	Accuracy	AUC-ROC [95% CI]	PR-AUC
ATS score (rules)	0.514	0.997	0.678	0.516	0.536 [0.51, 0.56]	0.544
TF-IDF cosine	0.574	0.784	0.663	0.591	0.612 [0.58, 0.64]	0.582
MiniLM cosine (pretrained)	0.537	0.876	0.666	0.550	0.579 [0.55, 0.61]	0.592
Bi-encoder cosine (fine-tuned)	0.571	0.887	0.695	0.601	0.633 [0.61, 0.66]	0.607
Logistic regression	0.563	0.897	0.692	0.590	0.648 [0.62, 0.67]	0.622
Gradient boosting	0.594	0.876	0.708	0.630	0.695 [0.67, 0.72]	0.679
MatchNet	**0.668**	**0.822**	**0.737**	**0.699**	**0.768 [0.74, 0.79]**	**0.757**

Table 5: Task B (human labels): test-set results (n = 1,759, 51% positive). Thresholds were chosen on the validation split by maximum F1.

Task A. MatchNet reaches F1 0.784 and AUC-ROC 0.862 (95% CI 0.85–0.88) on 2,352 test pairs, against 0.588 for the rule-based ATS score, 0.661 for the pretrained encoder and 0.790 for the fine-tuned bi-encoder alone. Accuracy by pair type: 0.883 on positives, 0.889 on random negatives, but only 0.510 on hard negatives. Almost all remaining errors are postings from a related occupation that the model accepts; this is the main weakness and the focus of the error analysis.

Task B. On human labels MatchNet reaches F1 0.737 (precision 0.668, recall 0.822) and AUC-ROC 0.768 (95% CI 0.74–0.79), against 0.536 for the ATS score and 0.579 for the pretrained encoder. Its mean score rises with the human grade: 0.344 for No Fit, 0.634 for Potential Fit and 0.687 for Good Fit; separating Good from No Fit alone gives AUC 0.788. The model therefore separates No Fit from the rest well but barely distinguishes Potential from Good Fit. The rule-based ATS score is close to chance on both tasks (AUC 0.588 and 0.536): it accepts nearly everything (recall above 0.99), which shows why a learned model is needed. Gradient boosting on the same features is the strongest baseline; MatchNet beats it on every threshold-free metric.

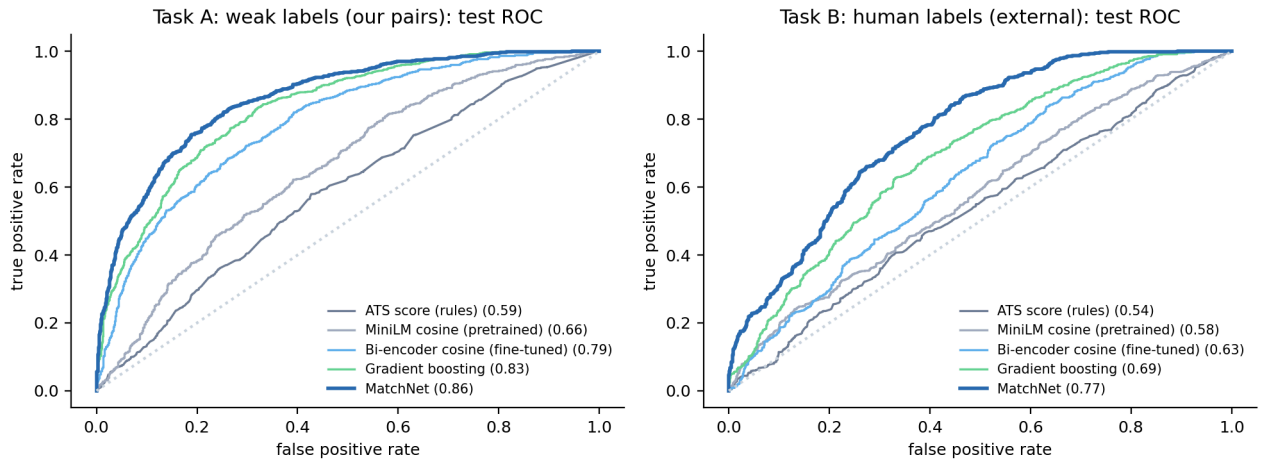


Figure 4: ROC curves on the test splits of Task A (left) and Task B (right).

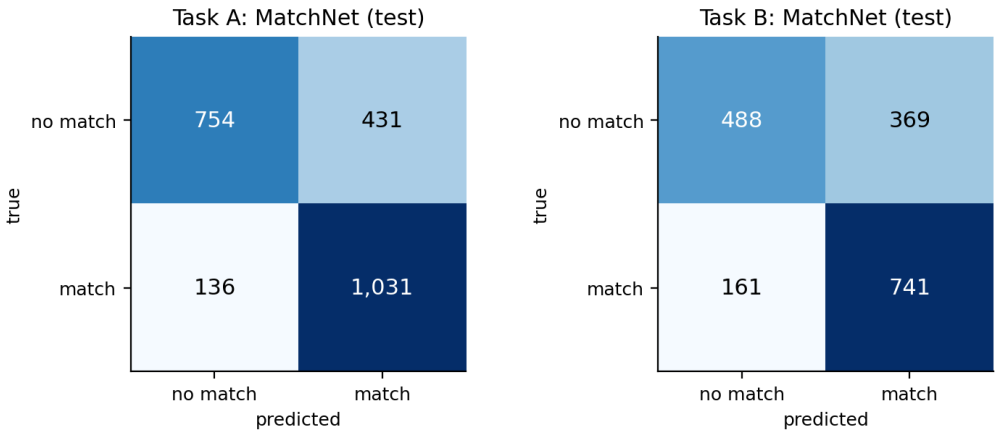


Figure 5: MatchNet confusion matrices on the test splits at the validation-selected threshold.

Top-3 recommendation. Table 6 evaluates the product flow end to end: for each test resume, all test postings are ranked and the top three are checked.

Ranker	Precision@3
TF-IDF cosine	0.406
MiniLM (pretrained)	0.417
Bi-encoder (fine-tuned)	0.634
Bi-encoder + MatchNet re-rank	0.641
Random ranking (expected)	0.050

Table 6: Top-3 recommendation on the test split: 395 resumes, each ranked against all 2,565 test postings with a known occupation. A recommended posting counts as relevant if it belongs to the resume's occupation.

Fine-tuning lifts precision@3 from 0.417 (pretrained) to 0.634, so roughly two of the three recommended postings are in the right occupation, against one in twenty for a random ranking (0.050). Re-ranking the top 20 with MatchNet adds only a little (0.641) on this occupation-level criterion; its value is in a match score that also weighs skills and experience, and in the explicit skill features shown to the user. The weakest occupations are Agriculture (0.00), Automobile (0.00), BPO (0.08); the strongest are Chef (1.00), HR (0.98), Accountant (0.96).

11 Error Analysis

Owner: Muaaz Fahad, module `src/error_analysis.py`.

The error analysis breaks MatchNet's test errors down by resume category, pair type and human fit grade, compares the features of correct and incorrect predictions, and inspects the most confident mistakes (Table 7).

Resume category	Precision	Recall	F1	Pairs
Pending: run `src/error_analysis.py`				

Table 7: MatchNet per resume category on the Task A test split.

- Worst categories by F1: *pending*. Best: *pending*.
- Task A error rate by pair type: positives *pending*, hard negatives *pending*, random negatives *pending*.
- Task B error rate by human grade: No Fit *pending*, Potential Fit *pending*, Good Fit *pending*.
- False positives / false negatives on Task A: 431 / 136; on Task B: 369 / 161.

12 Bias Check

Owner: Hanan Ishaque, module `src/bias_check.py`.

A matcher must not change its decision because of who the candidate is. We test this counterfactually: for 300 test pairs we create resume variants that differ in *one* attribute only and measure how much the MatchNet score moves. The resumes were anonymized in Deliverable 1, so the attribute is re-inserted explicitly:

- **gender (name)**: a male vs. a female first name with the same surname;
- **origin (name)**: a Pakistani vs. a Western name;
- **university**: a top-ranked vs. a less prestigious university with the same degree;
- **gender (pronouns)**: he/his/him vs. she/her/her in the body text.

Several name pairs are averaged per test. For each test we report the mean paired score difference with a bootstrap 95% interval, a Wilcoxon signed-rank test, and the share of pairs whose accept/reject decision flips at the MatchNet threshold (Table 8).

Counterfactual test	Mean Δ	95% CI	Mean |Δ|	p	Decisions flipped
Pending: run `src/bias_check.py`					

Table 8: Bias check: change in MatchNet score when only one attribute of the resume is swapped (variant B minus variant A).

Conclusion: *pending*

13 Limitations

- Task A labels are occupation-level, so Task A scores overstate how well the model judges seniority or specific skills; Task B is the more honest number but is a single external dataset.
- In Task B, test job descriptions are unseen but almost all test resumes also appear (with other postings) in training, so it measures generalization to new postings rather than new candidates.

- The bi-encoder was fine-tuned on occupation labels, which does not help human fit judgements (Section 6); fine-tuning on graded fit labels is future work.
- MatchNet accepts too many postings from related occupations (hard negatives, Section 10).
- Inputs are truncated to 256 word pieces; long resumes lose their later sections.
- Most live postings collected for Deliverable 1 have since expired; the pool must be refreshed (`make refresh`) before a demo.

14 Reproducibility and Model Weights

Code is in `src/` (models in `src/models/`), dependencies in `requirements.txt`. Weights are not stored in Git; they are published on the Hugging Face Hub and linked from the README (`huggingface.co/Suf-7/resumatch-models`), or rebuilt from scratch with:

- `make train` — bi-encoder grid search, pair matrices, MatchNet grid search;
- `make evaluate` — test metrics, top-3 retrieval, figures, report tables;
- `make report2` — this PDF.

15 Contribution Table

Table 9 is generated from `git log`, so it reflects the commit history exactly.

Git author	Commits	Lines	Main modules (by lines changed)
Muhammad Sufyan	7	+4,857 / −46	`report (build_pdf.py)`, `build_features.py`, `scrape_jobs.py`, `common.py`, `report (make_tables.py)`
Muaaz Fahad	0	+0 / −0	—
Hanan Ishaque	0	+0 / −0	—

Table 9: Contribution table generated from the Git history by `report/contributions.py` (generated files such as figures, PDFs and JSON statistics are excluded).

Member	Deliverable 2 responsibility
Muhammad Sufyan	Bi-encoder and MatchNet implementation, training and tuning, baselines, evaluation, retrieval evaluation, figures, report
Muaaz Fahad	Error analysis (`src/error_analysis.py`), Section 11
Hanan Ishaque	Bias check (`src/bias_check.py`), Section 12

Table 10: Module ownership.

16 Appendix: Full MatchNet Grid

LR	Batch	Dropout	H	A: val AUC	A: epoch	B: val AUC	B: epoch
1×10^{-3}	64	0.1	128	0.852	14	0.793	20
1×10^{-3}	64	0.1	256	0.858	14	0.795	14
1×10^{-3}	64	0.3	128	0.851	16	0.807	23
1×10^{-3}	64	0.3	256	0.860	14	0.796	11
1×10^{-3}	64	0.5	128	0.855	23	0.807	26
1×10^{-3}	64	0.5	256	0.858	14	0.806	14
1×10^{-3}	256	0.1	128	0.847	29	0.782	11
1×10^{-3}	256	0.1	256	0.846	30	0.780	9
1×10^{-3}	256	0.3	128	0.847	31	0.786	18
1×10^{-3}	256	0.3	256	0.853	31	0.789	16
1×10^{-3}	256	0.5	128	0.850	31	0.800	26
1×10^{-3}	256	0.5	256	0.857	38	0.793	16
3×10^{-4}	64	0.1	128	0.841	31	0.785	16
3×10^{-4}	64	0.1	256	0.843	31	0.779	16
3×10^{-4}	64	0.3	128	0.841	27	0.783	23
3×10^{-4}	64	0.3	256	0.845	31	0.778	16
3×10^{-4}	64	0.5	128	0.844	31	0.793	34
3×10^{-4}	64	0.5	256	0.847	36	0.787	16
3×10^{-4}	256	0.1	128	0.836	59	0.778	20
3×10^{-4}	256	0.1	256	0.837	46	0.770	20
3×10^{-4}	256	0.3	128	0.839	40	0.772	32
3×10^{-4}	256	0.3	256	0.838	37	0.771	26
3×10^{-4}	256	0.5	128	0.841	59	0.786	50
3×10^{-4}	256	0.5	256	0.840	38	0.784	44
1×10^{-4}	64	0.1	128	0.836	58	0.776	31
1×10^{-4}	64	0.1	256	0.836	51	0.773	20
1×10^{-4}	64	0.3	128	0.836	51	0.772	34
1×10^{-4}	64	0.3	256	0.837	40	0.773	25
1×10^{-4}	64	0.5	128	0.839	59	0.774	35
1×10^{-4}	64	0.5	256	0.838	40	0.781	52
1×10^{-4}	256	0.1	128	0.831	59	0.777	51
1×10^{-4}	256	0.1	256	0.831	59	0.773	38

LR	Batch	Dropout	H	A: val AUC	A: epoch	B: val AUC	B: epoch
1×10^{-4}	256	0.3	128	0.832	60	0.771	55
1×10^{-4}	256	0.3	256	0.833	59	0.771	38
1×10^{-4}	256	0.5	128	0.833	60	0.767	59
1×10^{-4}	256	0.5	256	0.835	59	0.774	51

Table 11: Full MatchNet grid (validation AUC and early-stopped epoch per task).